

First development changes understanding code block by block

To start this frontend part of the project off I connected my Next.js frontend to the real AWS backend I built during Week 3.

I installed **TanStack Query** and created `providers.tsx` so the application could use query caching, loading states, error handling and later mutations. I updated `layout.tsx` to wrap the application in the TanStack Query provider, created a shared TypeScript `Reading` type in `types/reading.ts`, and created `lib/api.ts` to contain the functions that communicate with my REST API. I initially tried calling API Gateway directly from the browser using the API URL and API key, but the browser sent a CORS preflight `OPTIONS` request which API Gateway rejected with a `403`.

So rather than changing my working Week 3 backend, I changed the design so the browser now calls a **Next.js Route Handler**, which acts as a server-side proxy to API Gateway. I moved the API URL and API key to server-only environment variables in `.env.local` (This `.env.local` file you will not see in git hub because it was git ignored, due to it holding both my API URL and the API gateway key), meaning the API key is no longer exposed to browser-side JavaScript. I created `app/api/backend/[...path]/route.ts` to forward GET, POST, PATCH and DELETE requests to the real AWS API.

I also encountered a `Missing Authentication Token` error because the API base URL was initially incorrect; after correcting it so that it ended at the `/dev` stage rather than including the full readings path, the frontend successfully returned real DynamoDB readings. Finally, I used TanStack Query in `app/readings/page.tsx` to fetch the readings for `BRANCH-001`, displaying proper loading, error and empty states while initially showing the successful response as raw JSON to prove the frontend-to-backend connection worked before building the final table interface.

(Most of this section was getting the backend to connect with the frontend, there was basic design that I made to show that the backend values and process were now being displayed on the front end)

1. `package.json` and `package-lock.json`

We did not manually write these, but they changed when you ran:

```
1 | npm install @tanstack/react-query
```

This installed TanStack Query into the frontend project.

`package.json` now records that your application depends on:

```
1 | @tanstack/react-query
```

while:

```
1 | package-lock.json
```

records the exact versions of the packages npm installed.

TanStack Query will eventually handle:

```
1 | GET requests
2 | Caching
3 | Loading states
4 | Error states
5 | Retries
6 | Refetching
7 | POST/PATCH/DELETE mutations
8 | Query invalidation
```

2. `app/providers.tsx`

We created this file from scratch.

```
1 | "use client";
```

This tells Next.js:

This component must run as a Client Component.

We need this because TanStack Query uses browser-side React features such as state.

Then:

```
1 | import {
2 |   QueryClient,
3 |   QueryClientProvider,
4 | } from "@tanstack/react-query";
```

This imports the two main TanStack Query pieces we need.

`QueryClient`

The `QueryClient` manages the application's query information.

You can think of it roughly as:

```
1 | QueryClient
2 |   ↓
3 | remembers API data
4 | remembers whether data is loading
5 | remembers errors
```

```
6 knows when data is stale
7 controls refetching
8 controls query invalidation
```

Then:

```
1 import {
2   useState,
3   type ReactNode,
4 } from "react";
```

`useState` is used to create the `QueryClient` once.

`ReactNode` is a TypeScript type used for:

```
1 children
```

Then we created:

```
1 type ProvidersProps = {
2   children: ReactNode;
3 };
```

This is another example of TypeScript typed props.

It says:

The `Providers` component expects something React can render inside it.

Then:

```
1 const [queryClient] = useState(
2   () => new QueryClient()
3 );
```

This creates the TanStack Query client.

Using `useState` here makes sure we don't unnecessarily create a new `QueryClient` every time React renders the component.

Finally:

```
1 return (
2   <QueryClientProvider client={queryClient}>
3     {children}
4   </QueryClientProvider>
5 );
```

This makes TanStack Query available to everything nested inside it.

Conceptually:

```
1 | QueryClientProvider
2 | |
3 | |   Home
4 | |   Manage
5 | |   Readings
```

That is why `/readings` can now use:

```
1 | useQuery()
```

and `/manage` will later be able to use:

```
1 | useMutation()
```

3. `app/layout.tsx`

Your original layout was mainly Next.js setup:

```
1 | import type { Metadata } from "next";
2 | import { Geist, Geist_Mono } from "next/font/google";
3 | import "./globals.css";
```

These handle:

- page metadata
- the Geist font
- your global CSS

We kept all of that.

The important new import was:

```
1 | import Providers from "./providers";
```

That brings in the TanStack Query provider we just created.

Your original body was essentially:

```
1 | <body className="min-h-full flex flex-col">
2 |   {children}
3 | </body>
```

We changed it to:

```
1 | <body className="min-h-full flex flex-col">
2 |   <Providers>
3 |     {children}
4 |   </Providers>
5 | </body>
```

This means every page in the application is now inside the TanStack Query provider.

So instead of:

```
1 layout
2   ↓
3 pages
```

we now have:

```
1 layout
2   ↓
3 Providers
4   ↓
5 TanStack Query
6   ↓
7 pages
```

4. types/reading.ts

We created this file so the whole frontend agrees on what a cold-chain reading looks like.

First:

```
1 export type ReadingStatus = "ok" | "excursion";
```

This is more specific than:

```
1 status: string
```

because TypeScript now knows that there are only two valid values:

```
1 ok
2 excursion
```

For example:

```
1 status: "ok"
```

is valid.

But:

```
1 status: "banana"
```

would be a TypeScript error.

Then:

```
1 export type Reading = {
2   readingId: string;
3   branchId: string;
4   recordedAt: string;
5   minTempC: number;
6   maxTempC: number;
7   status: ReadingStatus;
8   recordedBy: string;
```

```
9 alertRaisedAt?: string;  
10 };
```

This represents a reading returned by your real backend.

readingId

```
1 readingId: string;
```

The UUIDv4 generated by your backend.

Example:

```
1 ba9de34f-505c-41ae-abc5-63348320a3c4
```

branchId

```
1 branchId: string;
```

Identifies the branch.

Example:

```
1 BRANCH-001
```

recordedAt

```
1 recordedAt: string;
```

The backend sends the timestamp as an ISO date string.

For example:

```
1 2026-09-07T09:00:00.000Z
```

Later our table will turn that into readable date and time columns.

Temperature values

```
1 minTempC: number;  
2 maxTempC: number;
```

These must be numbers.

Status

```
1 status: ReadingStatus;
```

Which means:

```
1 ok
```

```
2 or
3 excursion
```

Recorded by

```
1 recordedBy: string;
```

The person who recorded the temperature.

Alert time

```
1 alertRaisedAt?: string;
```

The `?` is important.

It means:

This field is optional.

Normal readings may not have an alert at all.

An excursion may eventually contain:

```
1 alertRaisedAt
```

after your SNS → SQS → excursion Lambda process has run.

5. Original `lib/api.ts`

This file is the layer between your React components and the API.

Instead of putting `fetch()` inside every page, we centralise API communication here.

Originally we had something like:

```
1 const API_BASE_URL =
2   process.env.NEXT_PUBLIC_API_BASE_URL;
3
4 const API_KEY =
5   process.env.NEXT_PUBLIC_API_KEY;
```

The problem was the word:

```
1 NEXT_PUBLIC_
```

In Next.js, anything beginning with that is available in browser JavaScript.

That meant our architecture was:

```
1 Browser
2   ↓
```

and the API key would also be sent from the browser.

That led us directly into the CORS problem.

6. `.env.local`

Originally we used:

```
1 NEXT_PUBLIC_API_BASE_URL=...
2 NEXT_PUBLIC_API_KEY=...
```

After finding the CORS problem, we deliberately changed this to:

```
1 API_BASE_URL=https://...execute-api.eu-north-1.amazonaws.com/dev
2 API_KEY=...
```

The important difference is:

```
1 NEXT_PUBLIC_API_KEY
```

can be accessed by frontend/browser JavaScript.

Whereas:

```
1 API_KEY
```

is only available to server-side Next.js code.

So we changed the architecture to:

```
1 Browser
2   ↓
3 Next.js server
4   ↓
5 API Gateway
```

This is better for your current application because the browser doesn't need to know the API key.

Also, `.env.local` should remain ignored by Git.

You should **never commit the actual API key to GitHub**.

7. `app/api/backend/[...path]/route.ts`

This was the major change that fixed our CORS problem without changing your backend.

First, the unusual folder:

```
1 [...path]
```

is a Next.js **catch-all dynamic route**.

For example:

```
1 /api/backend/branches/BRANCH-001/readings
```

becomes:

```
1 path = [  
2   "branches",  
3   "BRANCH-001",  
4   "readings"  
5 ]
```

Whereas:

```
1 /api/backend/branches/BRANCH-001/readings/UUID
```

becomes:

```
1 path = [  
2   "branches",  
3   "BRANCH-001",  
4   "readings",  
5   "UUID"  
6 ]
```

So we don't need five separate proxy routes.

The route context

We created:

```
1 type RouteContext = {  
2   params: Promise<{  
3     path: string[];  
4   }>;  
5 };
```

This tells TypeScript what Next.js will give our route.

`path` is an array because `[...path]` can contain multiple URL sections.

The `forwardRequest()` function

This is the heart of the proxy.

```
1 async function forwardRequest(  
2   request: Request,  
3   context: RouteContext  
4 )
```

Rather than writing the same code four times for GET, POST, PATCH and DELETE, they all call this shared function.

Reading the environment variables

```
1 | const API_BASE_URL = process.env.API_BASE_URL;  
2 | const API_KEY = process.env.API_KEY;
```

These values are read on the **Next.js server**.

The browser doesn't receive them directly.

Then we check that they actually exist:

```
1 | if (!API_BASE_URL) {
```

and:

```
1 | if (!API_KEY) {
```

If something is missing, the server returns a useful **500** response rather than failing mysteriously.

Reading [...path]

```
1 | const { path } = await context.params;
```

This retrieves the requested path.

For example:

```
1 | branches/BRANCH-001/readings
```

Keeping query parameters

```
1 | const incomingUrl = new URL(request.url);
```

This reads the URL that came into Next.js.

Then:

```
1 | incomingUrl.search
```

keeps query strings.

This will matter if we later use things such as:

```
1 | ?from=...  
2 | &to=...
```

Building the AWS URL

```
1 const backendUrl =
2   `${API_BASE_URL}/${path.join("/")}` +
3   incomingUrl.search;
```

Suppose:

```
1 API_BASE_URL =
2 https://abc.execute-api.eu-north-1.amazonaws.com/dev
```

and:

```
1 path = [
2   "branches",
3   "BRANCH-001",
4   "readings"
5 ];
```

The result becomes:

```
1 https://abc.execute-api.eu-north-1.amazonaws.com/dev/branches/BRANCH-
001/readings
```

That is your real API Gateway endpoint.

Adding the API key

```
1 const headers: HeadersInit = {
2   "x-api-key": API_KEY,
3 };
```

This is where the Next.js server adds the API key.

So now:

```
1 Browser
```

does not add it.

Instead:

```
1 Next.js server
```

adds it before calling AWS.

Handling POST and PATCH bodies

We have:

```
1 if (
2   request.method === "POST" ||
3   request.method === "PATCH"
4 )
```

because these requests send JSON data.

Then:

```
1 headers["Content-Type"] = "application/json";  
2 body = await request.text();
```

This takes the JSON request sent by the frontend and passes it onwards to API Gateway.

Calling AWS

Then:

```
1 const response = await fetch(  
2   backendUrl,  
3   {  
4     method: request.method,  
5     headers,  
6     body,  
7     cache: "no-store",  
8   }  
9 );
```

This is the point where Next.js actually talks to your backend.

Notice:

```
1 method: request.method
```

This means if the browser calls:

```
1 GET
```

Next.js sends:

```
1 GET
```

If it receives:

```
1 PATCH
```

it sends:

```
1 PATCH
```

etc.

Returning AWS's response

We read:

```
1 const responseBody = await response.text();
```

Then return it:

```
1 return new Response(  
2   responseBody,  
3   {  
4     status: response.status,  
5     headers: response.headers,  
6   }  
7 );
```

```
2   responseBody,  
3   {  
4     status: response.status,  
5     ...  
6   }  
7 );
```

So if Lambda/API Gateway returns:

```
1 | 200
```

the frontend gets `200` .

If AWS returns:

```
1 | 400
```

the frontend gets `400` .

If AWS returns:

```
1 | 404
```

the frontend gets `404` .

This is important because later we can show the user the **real validation errors from your backend**.

GET, POST, PATCH and DELETE exports

At the bottom:

```
1 | export async function GET(...)
```

```
1 | export async function POST(...)
```

```
1 | export async function PATCH(...)
```

```
1 | export async function DELETE(...)
```

Each one simply calls:

```
1 | forwardRequest()
```

That means one proxy handles all four methods.

8. Final `lib/api.ts`

After creating the proxy, we changed this file again.

Instead of:

```
1 | Browser → AWS
```

we now want:

```
1 | Browser → Next.js proxy
```

So we created:

```
1 | const FRONTEND_API_BASE = "/api/backend";
```

This means the browser only calls your own Next.js app.

CreateReadingInput

```
1 | export type CreateReadingInput = {  
2 |   branchId: string;  
3 |   recordedAt: string;  
4 |   minTempC: number;  
5 |   maxTempC: number;  
6 |   recordedBy: string;  
7 | };
```

This describes what Page 2 will eventually send when creating a reading.

Notice there is **no** `readingId`.

That is deliberate because your backend generates the UUID.

There is also no:

```
1 | status  
2 | alertRaisedAt
```

because those are controlled by the backend.

UpdateReadingInput

```
1 | export type UpdateReadingInput = {  
2 |   minTempC?: number;  
3 |   maxTempC?: number;  
4 |   recordedBy?: string;  
5 | };
```

Everything has:

```
1 | ?
```

because PATCH is a **partial update**.

For example, we might only update:

```
1 | {  
2 |   "maxTempC": 9  
3 | }
```

without changing everything else.

ApiError

We created:

```
1 export class ApiError extends Error
```

This lets us carry both:

```
1 error message
2 HTTP status
```

For example:

```
1 400
2 Maximum temperature cannot be lower than minimum temperature
```

instead of reducing everything to:

```
1 Something went wrong
```

That directly helps with the ticket requirement to show useful API validation errors.

```
apiRequest<T>()
```

This is our reusable fetch helper:

```
1 async function apiRequest<T>(
```

The `<T>` is TypeScript.

It means different calls can tell this function what type of data they expect back.

For example:

```
1 apiRequest<Reading[]>
```

means:

I expect an array of readings.

Whereas:

```
1 apiRequest<Reading>
```

means:

I expect one reading.

It now calls:

```
1 fetch(  
2   `${FRONTEND_API_BASE}${path}`  
3 )
```

For example:

```
1 /api/backend/branches/BRANCH-001/readings
```

rather than the AWS URL directly.

Error handling

We check:

```
1 if (!response.ok)
```

Then try to extract:

```
1   errorData.message
```

or:

```
1   errorData.error
```

from the backend.

This will eventually allow us to show something useful like:

```
1 Recorded At cannot be in the future.
```

instead of:

```
1 Request failed.
```

GET branch

```
1 export async function getBranchReadings(  
2   branchId: string  
3 ): Promise<Reading[]>
```

This calls:

```
1 GET  
2 /branches/{branchId}/readings
```

and expects:

```
1 Reading[]
```

which means an array of readings.

This is what your current `/readings` page is using successfully.

GET individual

```
1 | getReadingById()
```

calls:

```
1 | GET  
2 | /branches/{branchId}/readings/{readingId}
```

We will use this for your **Individual Reading** selector.

POST

```
1 | createReading()
```

takes the form data and sends:

```
1 | POST  
2 | /branches/{branchId}/readings
```

Notice:

```
1 | const { branchId, ...body } = input;
```

This removes `branchId` from the JSON body because it belongs in the URL.

So:

```
1 | input
```

might contain:

```
1 | branchId  
2 | recordedAt  
3 | minTempC  
4 | maxTempC  
5 | recordedBy
```

but the HTTP request becomes:

```
1 | URL:  
2 | branches/BRANCH-001/readings  
3 |  
4 | BODY:  
5 | recordedAt  
6 | minTempC  
7 | maxTempC  
8 | recordedBy
```

which matches your backend design.

PATCH

```
1 | updateReading()
```

calls:

```
1 PATCH
2 /branches/{branchId}/readings/{readingId}
```

This is the extra REST functionality you added during Week 3.

DELETE

```
1 deleteReading()
```

calls:

```
1 DELETE
2 /branches/{branchId}/readings/{readingId}
```

This gives Page 2 access to your backend DELETE operation.

9. app/page.tsx

We put a temporary real homepage here so `/` had a valid React component.

It currently contains:

```
1 Cold Chain Operations Portal
```

and two navigation buttons:

```
1 Manage Readings
2 View Readings
```

These use:

```
1 <Link href="/manage">
```

and:

```
1 <Link href="/readings">
```

This isn't the final Encon-styled homepage yet.

We only built enough to establish the three-page navigation structure:

```
1 /
2 └─ /manage
3 └─ /readings
```

Later we will make this look much more like the Encon design screenshots you provided.

10. app/readings/page.tsx

This is currently the most important page because it's the first one talking to the backend.

At the top:

```
1 | "use client";
```

We need this because the page uses:

```
1 | useState
2 | useQuery
```

Branch state

```
1 | const [branchId, setBranchId] =
2 |   useState("BRANCH-001");
```

This stores which branch the user wants to view.

We start with:

```
1 | BRANCH-001
```

because that's the branch we have been using throughout the project.

TanStack Query

This is the main new part:

```
1 | const {
2 |   data: readings,
3 |   isLoading,
4 |   isError,
5 |   error,
6 |   refetch,
7 | } = useQuery({
```

TanStack Query gives us several useful values automatically.

readings

```
1 | The data returned by AWS
```

isLoading

```
1 | Is the request currently loading?
```

isError

```
1 | Did it fail?
```

error

```
1 | What error occurred?
```

refetch

```
1 | Run the request again
```

Query key

```
1 | queryKey: ["readings", branchId],
```

This is important.

TanStack Query identifies the cached data using this key.

For:

```
1 | BRANCH-001
```

the cache key is effectively:

```
1 | readings + BRANCH-001
```

If later you choose:

```
1 | BRANCH-002
```

TanStack Query knows those are different sets of data.

Query function

```
1 | queryFn: () =>  
2 |   getBranchReadings(branchId),
```

This says:

When this query runs, call the function we built in `lib/api.ts`.

That produces:

```
1 | useQuery  
2 |   ↓  
3 | getBranchReadings()  
4 |   ↓  
5 | /api/backend  
6 |   ↓  
7 | route.ts  
8 |   ↓  
9 | API Gateway  
10 |  ↓  
11 | Lambda  
12 |   ↓  
13 | DynamoDB
```

Which is exactly what your successful screenshot proved.

Loading state

We created:

```
1 {isLoading && (  
2   <p>  
3     Loading temperature readings...  
4   </p>  
5 )}
```

That's why you saw:

```
1 Loading temperature readings...
```

before the data appeared.

This proves one of the ticket states already works.

Error state

We also created:

```
1 {isError && (  
2   <p>  
3     Loading temperature readings...  
4   </p>  
5 )}
```

and displayed the actual error message.

That is how we saw:

```
1 Load failed
```

and later:

```
1 Missing Authentication Token
```

Those errors were actually very helpful while debugging.

Empty state

We created:

```
1 readings?.length === 0
```

which displays:

```
1 No readings recorded for this branch yet.
```

This is directly required by ticket 4.2.

Temporary raw JSON

Finally:

```
1 JSON.stringify(readings, null, 2)
```

produced the big block of real AWS data you just saw.

That was deliberate.

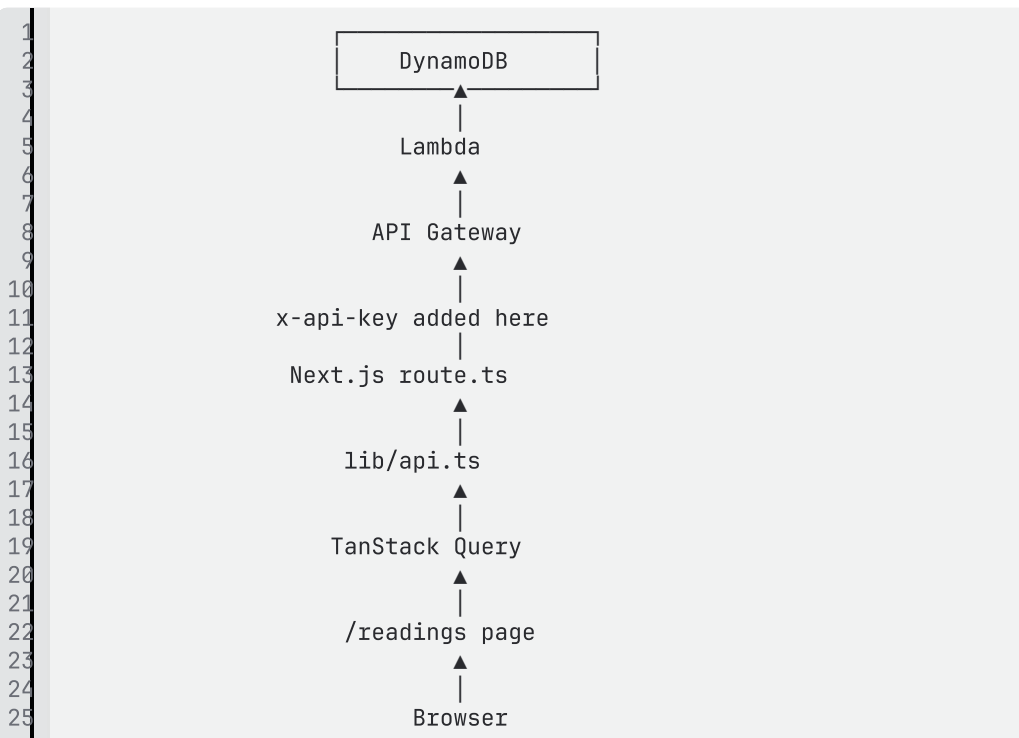
Instead of immediately writing a complex table and then wondering whether the API or the table was broken, we first proved:

```
1 API works ✓
2 Data arrives ✓
3 TypeScript accepts it ✓
4 TanStack Query works ✓
5 Proxy works ✓
6 Backend works ✓
```

Now we can build the table knowing the connection underneath it is already functioning.

Where we are now

The frontend currently works like this:



And the biggest milestone is that **the browser is now displaying real readings from the AWS backend you built in Weeks 2–3.**